PySpark案例实战

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 了解什么是Spark、PySpark
2. 了解为什么学习PySpark
3. 了解课程是如何和大数据开发方向进行衔接

Spark是什么

定义：Apache Spark是用于**大规模数据**（large-scala data）处理的**统一（unified）分析引擎**。

Apache Spark™

Apache Spark is a lightning-fast **unified analytics engine** for big data and machine learning. It was originally developed at UC Berkeley in 2009.

简单来说，Spark是一款分布式的计算框架，用于调度成百上千的服务器集群，计算TB、PB乃至EB级别的海量数据

Python On Spark

Spark作为全球顶级的分布式计算框架，支持众多的编程语言进行开发。

而Python语言，则是Spark重点支持的方向。

Spark Release 3.0.0

Apache Spark 3.0.0 is the first release of the 3.x line. The vote passed on the 10th of June, 2020. This release is based on git tag `v3.0.0` which includes all commits up to June 10. Apache Spark 3.0 builds on many of the innovations from Spark 2.x, bringing new ideas as well as continuing long-term projects that have been in development. With the help of tremendous contributions from the open-source community, this release resolved more than 3400 tickets as the result of contributions from over 440 contributors.

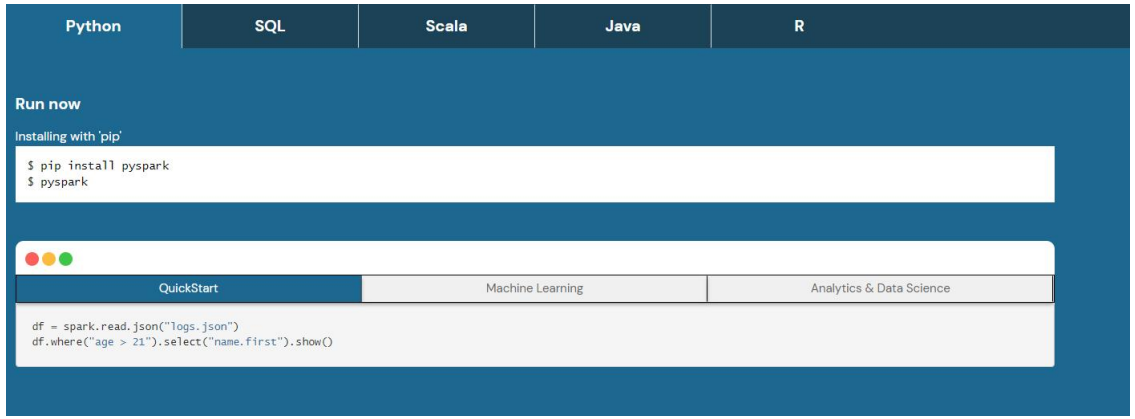
This year is Spark's 10-year anniversary as an open source project. Since its initial release in 2010, Spark has grown to be one of the most active open source projects. Nowadays, Spark is the de facto unified engine for big data processing, data science, machine learning and data analytics workloads.

Python目前是Spark中使用最广泛的语言，PySpark在PyPI每月有500万下载量

Spark SQL is the top active component in this release. 46% of the resolved tickets are for Spark SQL. These enhancements benefit all the higher-level libraries, including structured streaming and MLlib, and higher level APIs, including SQL and DataFrames. Various related optimizations are added in this release. In TPC-DS 30TB benchmark, Spark 3.0 is roughly two times faster than Spark 2.4.

Python is now the most widely used language on Spark. PySpark has more than 5 million monthly downloads on PyPI the Python Package Index. This release improves its functionalities and usability, including the pandas UDF API redesign with Python type hints, new pandas UDF types, and more Pythonic error handling.

图片来源: Spark官网

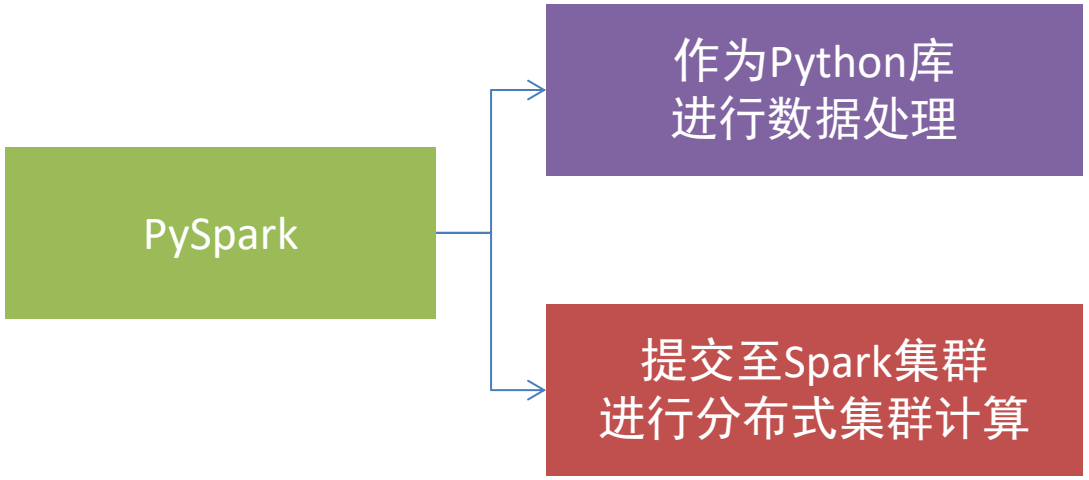


PySpark

Spark对Python语言的支持，重点体现在，Python第三方库：PySpark之上。

PySpark是由Spark官方开发的Python语言第三方库。

Python开发者可以使用pip程序快速的安装PySpark并像其它三方库那样直接使用。



为什么要学习PySpark技术呢？

Why PySpark

Python应用场景和就业方向是十分丰富的，其中，最为亮点的方向为：

大数据开发 和 人工智能

Why PySpark

招聘中

大数据工程师 20-40K·14薪

北京 · 1-3年 · 本科

感兴趣 立即沟通

鲍先生 HRBP · 刚刚活跃

职位描述

- Responsibilities
- 负责业务数据仓库、大数据处理平台的建设和维护
- 数据ETL的设计、开发与优化
- 负责建设、优化离线和实时大数据处理流程和系统
- 深刻理解业务数据需求，负责数据治理工作，提升数据质量和易用
- requirements
- 对大数据开发有系统的实践，有分布式计算和海量数据处理应用经
- 熟悉Hadoop、Hive、Spark、ES、Kafka、Flume、Flink、Kubene
- 较强的编码能力，熟练掌握Java/Shell/Python/Scala/Glang等至少
- 具有大数据处理经验，有数据建模或大型数据仓库项目实施者优先
- 有AWS使用经验优先

大数据开发工程师

更新于 今天

2万-3万 行业薪资对比

北京-丰台区 | 5-10年 | 大专 | 全职

职位描述

数据库开发

岗位职责：

- 3年以上数据处理开发经验（ETL等）
- 熟悉spark&yarn&Hadoop是一大优势
- 1年以上Linux环境开发经验
- 熟悉分布式计算和分布式存储。
- 精通Python或Scala/Java编程经验者优先考虑

大数据工程师 25-50K

北京 · 1年以内 · 本科

感兴趣 立即沟通

杨先生 大数据和AI训练平台架构 · 刚刚活跃

微信扫码分享 感兴

职位描述

岗位职责

1.参与数据平台的设计开发，构建高效、健壮的数据计算系统，保证高可用、稳定、低延迟的优质服务体验，

2.承担大数据平台服务组件的搭建和维护，优化现有的技术框架，形成配置化、可复用的数据技术能力；

3.通过数据模型跟踪和分析业务效果，为业务提出优化解决方案。

岗位要求：

1.本科及以上学历，2年及以上大数据项目开发经验或数据中台产品研发经验；

2. 精通Java/Python/Scala等一门语言，精通SQL，熟悉Hadoop、Spark、Hive、Flink、Kafka、ES等主流的大数据技术；

3. 有数据采集、作业调度、监控告警、数据治理等平台设计和开发经验的优先；

4. 有集成BI系统或数据中台使用者优先；

5. 擅于沟通和解决问题，对于业务可以进行抽象总结，推动并优化业务发展。

大数据算法/架构工程师(... 30-45K

北京 · 3-5年 · 本科

感兴趣 立即沟通

栗女士 HR · 刚刚活跃

职位描述

岗位职责：

1、负责数据仓库和大数据处理模块的架构设计和开发；

2、负责基于Spark技术的海量数据的处理、分析、统计、挖掘工作；

3、基于Spark框架的数据仓库的设计，开发，维护；

4、根据需求使用SparkStreaming和Spark SQL进行数据处理、查询、统计等工作,能

的留存、行为预测。

5、对于使用TiSpark进行优化，了解Tidb的使用。

任职要求：

1、熟悉Spark相关技术，3年以上Spark开发经验，有实时数据处理经验优先；

2、熟悉SparkStreaming、Flink和Spark SQL、Spark Mllib

3、熟练使用java、python语言等

大数据是Python的高薪就业方向，Spark（PySpark）技术是大数据的核心技术栈

衔接大数据方向

只会Python就学大数据，能学会吗？

课程中，不会涉及到分布式等大数据相关理论，仅使用PySpark作为普通的Python第三方库进行使用。

会Python，就能学会。

黑马程序员最新Python教程，8天python从入门到精通，学python看这套就够了

黑马程序员 · 8-8

2022最新黑马程序员大数据Hadoop入门视频教程，最适合零基础自学的大数据...

黑马程序员 · 2021-12-9

黑马程序员Spark全套视频教程，4天spark3.2快速入门到精通，全网首套基于Pyt...

衔接大数据方向

黑马程序员 LVE UP

置顶 教程配套资料关注微信公众号：黑马程序员，回复：Python
点赞，转发+收藏，关注黑马不迷茫
Python学习q群284710916，告别孤单，共同进步！
2022版Python+大数据最新最全学习路线图
2022Python+大数据学习路线图
资源下载问题
黑马程序员教程资源下载指南

Python+大数据开发
MySQL数据库：2022黑马程序员最新MySQL知识精讲+mysql实战案例_零基础mysql数据库入门到高级全套教程
Hadoop入门：2022最新黑马程序员大数据Hadoop入门视频教程，最适合零基础自学的大数据Hadoop教程
Hive数仓项目：黑马程序员大数据项目实战教程_大数据企业级离线数据仓库，在线教育项目实战（Hive数仓项目完整流程）

PB内存计算
Python入门：黑马程序员最新Python教程，8天python从入门到精通，学python看这套就够了
Python编程进阶：黑马程序员Python高级语法进阶教程_python多任务及网络编程，从零搭建网站全套教程
spark3.2从基础到精通：黑马程序员Spark全套视频教程，4天spark3.2快速入门到精通，全网首套基于Python语言的spark教程
Hive+Spark离线数仓工业项目实战：全网首次披露大数据Spark离线数仓工业项目实战，Hive+Spark构建企业级大数据平台

总结

1. 什么是Spark、什么是PySpark

- Spark是Apache基金会旗下的顶级开源项目，用于对海量数据进行大规模分布式计算。
- PySpark是Spark的Python实现，是Spark为Python开发者提供的编程入口，用于以Python代码完成Spark任务的开发
- PySpark不仅可以作为Python第三方库使用，也可以将程序提交的Spark集群环境中，调度大规模集群进行执行。

2. 为什么要学习PySpark？

大数据开发是Python众多就业方向中的明星赛道，薪资高岗位多，Spark（PySpark）又是大数据开发中的核心技术

总结

3. 如何衔接大数据开发方向？

课程以Python第三方库的视角学习使用PySpark，不涉及专业大数据知识，所以完全可以学会

想要深入大数据开发领域：

黑马程序员最新Python教程，8天python从入门到精通，学python看这套就够了

黑马程序员 · 8-8

2022最新黑马程序员大数据Hadoop入门视频教程，最适合零基础自学的大数据...

黑马程序员 · 2021-12-9

黑马程序员Spark全套视频教程，4天spark3.2快速入门到精通，全网首套基于Pyt...

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握PySpark库的安装
2. 掌握PySpark执行环境入口对象的构建
3. 理解PySpark的编程模型

PySpark库的安装

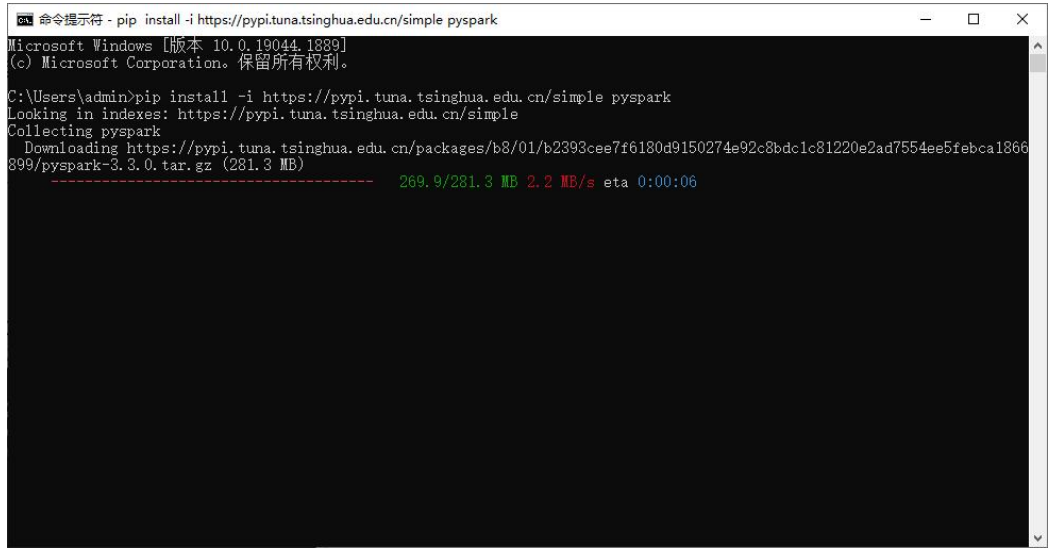
同其它的Python第三方库一样，PySpark同样可以使用pip程序进行安装。

在”CMD”命令提示符程序内，输入：

```
pip install pyspark
```

或使用国内代理镜像网站（清华大学源）

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple pyspark
```



构建PySpark执行环境入口对象

想要使用PySpark库完成数据处理，首先需要构建一个执行环境入口对象。

PySpark的执行环境入口对象是：类 SparkContext 的类对象

```
# 导包
from pyspark import SparkConf, SparkContext

# 创建SparkConf类对象
conf = SparkConf().setMaster("local[*]").\
    setAppName("test_spark_app")
# 基于SparkConf类对象创建SparkContext类对象
sc = SparkContext(conf=conf)

# 打印PySpark的运行版本
print(sc.version)

# 停止SparkContext对象的运行（停止PySpark程序）
sc.stop()
```

```
22/09/16 17:30:59 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unse
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
22/09/16 17:30:59 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
3.3.0

Process finished with exit code 0
```

PySpark的编程模型

SparkContext类对象，是PySpark编程中一切功能的入口。

PySpark的编程，主要分为如下三大步骤：

数据输入
通过SparkContext类对象的成员方法
完成数据的读取操作
读取后得到RDD类对象

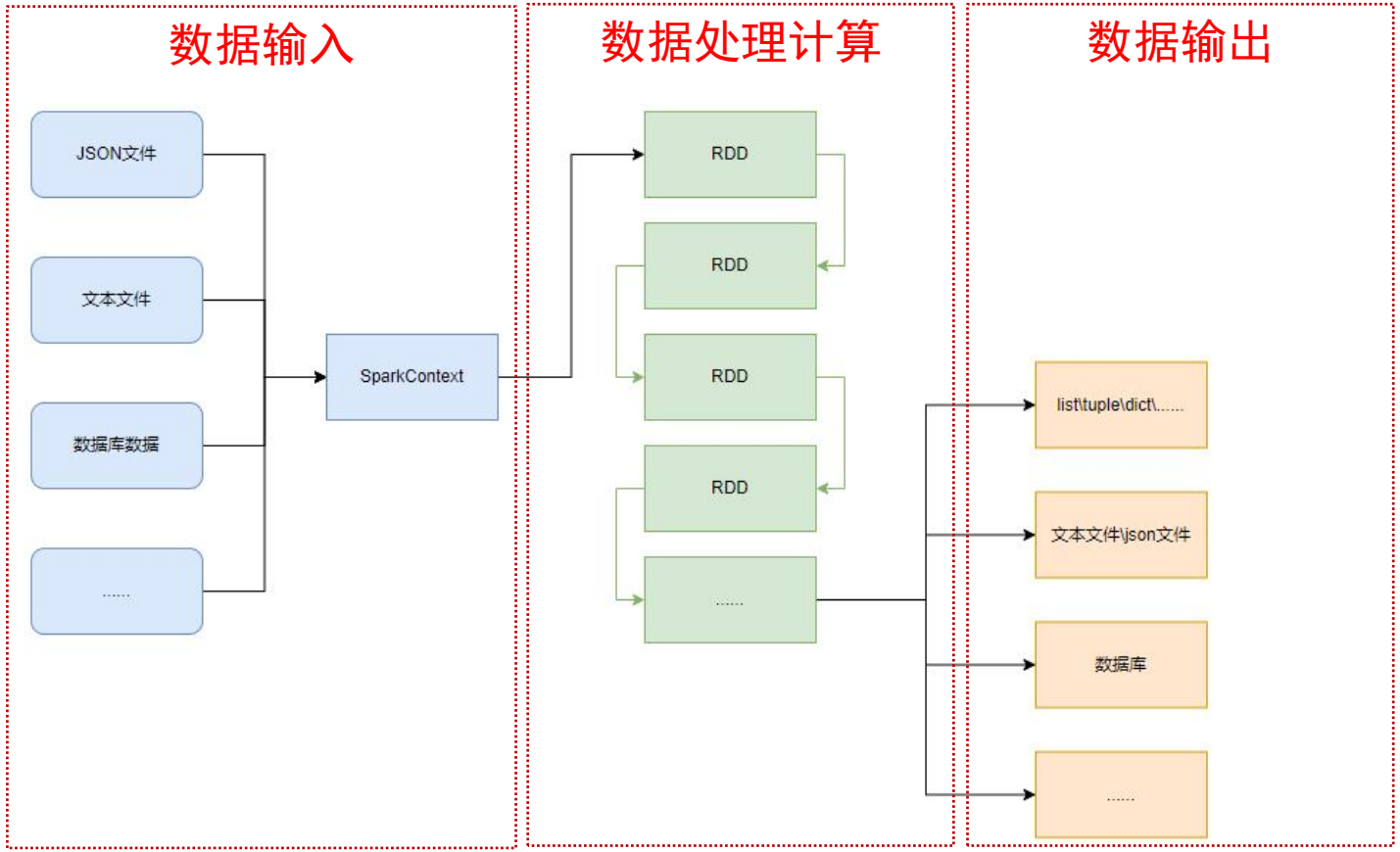
→

数据处理计算
通过RDD类对象的成员方法
完成各种数据计算的需求

→

数据输出
将处理完成后的RDD对象
调用各种成员方法完成
写出文件、转换为list等操作

PySpark的编程模型



- 通过SparkContext对象，完成数据输入
- 输入数据后得到RDD对象，对RDD对象进行迭代计算
- 最终通过RDD对象的成员方法，完成数据输出工作

总结

1. 如何安装PySpark库

pip install pyspark

2. 为什么要构建SparkContext对象作为执行入口

PySpark的功能都是从SparkContext对象作为开始

3. PySpark的编程模型是？

- 数据输入：通过SparkContext完成数据读取
- 数据计算：读取到的数据转换为RDD对象，调用RDD的成员方法完成计算
- 数据输出：调用RDD的数据输出相关成员方法，将结果输出到list、元组、字典、文本文件、数据库等

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 理解RDD对象
2. 掌握PySpark数据输入的2种方法

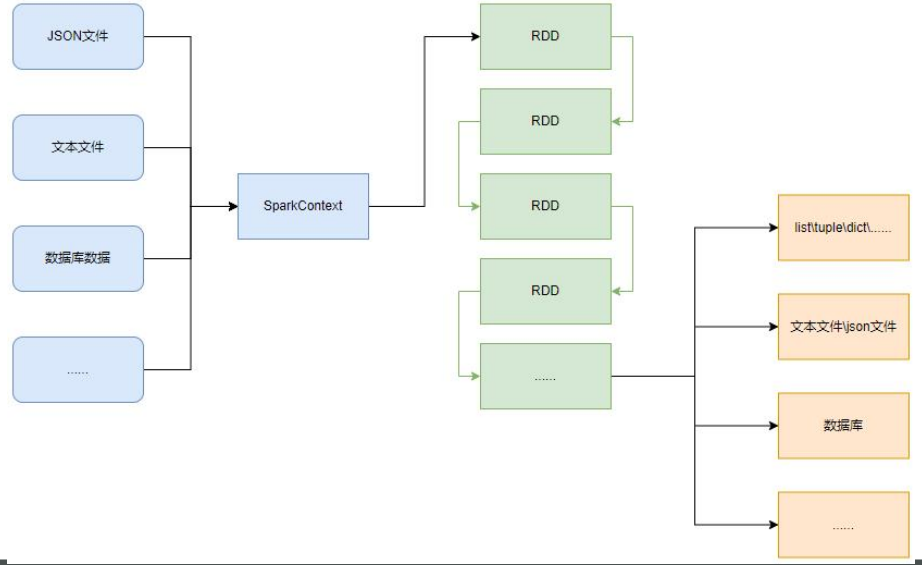
RDD对象

如图可见，PySpark支持多种数据的输入，在输入完成后，都会得到一个：RDD类的对象

RDD全称为：弹性分布式数据集（Resilient Distributed Datasets）

PySpark针对数据的处理，都是以RDD对象作为载体，即：

- 数据存储在RDD内
- 各类数据的计算方法，也都是RDD的成员方法
- RDD的数据计算方法，返回值依旧是RDD对象



PySpark的编程模型（左图）可以归纳为：
- 准备数据到RDD -> RDD迭代计算 -> RDD导出为list、文本文件等
- 即：源数据 -> RDD -> 结果数据

Python数据容器转RDD对象

PySpark支持通过SparkContext对象的parallelize成员方法，将：

- list
- tuple
- set
- dict
- str

转换为PySpark的RDD对象

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setMaster("local[*]").\
    setAppName("test_spark_app")
sc = SparkContext(conf=conf)

rdd = sc.parallelize(数据容器对象)

# 输出RDD的内容
print(rdd.collect())
```

注意：

- 字符串会被拆分出1个个的字符，存入RDD对象
- 字典仅有key会被存入RDD对象

读取文件转RDD对象

PySpark也支持通过SparkContext入口对象，来读取文件，来构建出RDD对象。

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setMaster("local[*]").\
    setAppName("test_spark_app")
sc = SparkContext(conf=conf)

rdd = sc.textFile(文件路径)

# 打印RDD内容
print(rdd.collect())
```

总结

1. RDD对象是什么？为什么要使用它？

RDD对象称之为分布式弹性数据集，是PySpark中数据计算的载体，它可以：

- 提供数据存储
- 提供数据计算的各类方法
- 数据计算的方法，返回值依旧是RDD（RDD迭代计算）

后续对数据进行各类计算，都是基于RDD对象进行

2. 如何输入数据到Spark（即得到RDD对象）

- 通过SparkContext的parallelize成员方法，将Python数据容器转换为RDD对象
- 通过SparkContext的textFile成员方法，读取文本文件得到RDD对象

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的map方法

map方法

PySpark的数据计算，都是基于RDD对象来进行的，那么如何进行呢？

自然是依赖，RDD对象内置丰富的：成员方法（算子）

map算子

功能: map算子, 是将RDD的数据 一条条处理(处理的逻辑 基于map算子中接收的处理函数), 返回新的RDD

语法:

```
rdd.map(func)
# func :   f:(T) → U
# f:  表示这是一个函数(方法)
# (T) → U 表示的是方法的定义:
#   () 表示传入参数,  (T) 表示 传入1个参数  , () 表示没有传入参数
# T 是泛型的代称, 在这里表示 任意类型
# U 也是泛型代称, 在这里表示 任意类型

# → U  表示返回值

# (T) → U 总结起来的意思是: 这是一个方法, 这个方法接受一个参数传入, 传入参数类型不限.  返回一个返回值, 返回值类型不限.

# (A) → A 总结起来的意思是: 这是一个方法, 这个方法接受一个参数传入, 传入参数类型不限.  返回一个返回值, 返回值和传入参数类型一致
```

map算子

演示代码:

```
# coding:utf8
# 演示rdd的map算子

from pyspark import SparkConf, SparkContext

if __name__ == '__main__':
    # 0. 构建Spark执行环境
    conf = SparkConf().setAppName("create rdd"). \
        setMaster("local[*]")
    sc = SparkContext(conf=conf)

    rdd = sc.parallelize([1, 2, 3, 4, 5])


    # f:(T) -> U
    def map_func(data):
        return data * 10


    print(rdd.map(map_func).collect())

    # 匿名Lambda的方式:
    print(rdd.map(lambda data: data * 10).collect())
```

对于传入参数的Lambda表达式:

传入方法作为传参的时候, 可以选择

1. 定义方法, 传入其方法名
2. 使用lambda 匿名方法的方式

一般, 如果方法体可以一行写完, 用lambda方便

如果方法体复杂 就直接定义方法更方便

总结

1. map算子（成员方法）

- 接受一个处理函数，可用lambda表达式快速编写
- 对RDD内的元素逐个处理，并返回一个新的RDD

2. 链式调用

- 对于返回值是新RDD的算子，可以通过链式调用的方式多次调用算子。

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的flatMap方法

flatMap算子

功能: 对rdd执行map操作, 然后进行 解除嵌套 操作.

解除嵌套:

```
# 嵌套的list
lst = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# 如果解除了嵌套
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

演示代码:

```
# coding:utf8
# 演示rdd的flatMap算子

from pyspark import SparkConf, SparkContext

if __name__ == '__main__':
    # 0. 构建Spark执行环境
    conf = SparkConf().setAppName("create rdd"). \
        setMaster("local[*]")
    sc = SparkContext(conf=conf)

    rdd = sc.parallelize(["a b c", "a c e", "e c a"])

    # 按照空格切分数据后，解除嵌套
    print(rdd.flatMap(lambda x: x.split(" ")).collect())
```

总结

1. flatMap算子

- 计算逻辑和map一样
- 可以比map多出，解除一层嵌套的功能

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的reduceByKey方法

reduceByKey算子

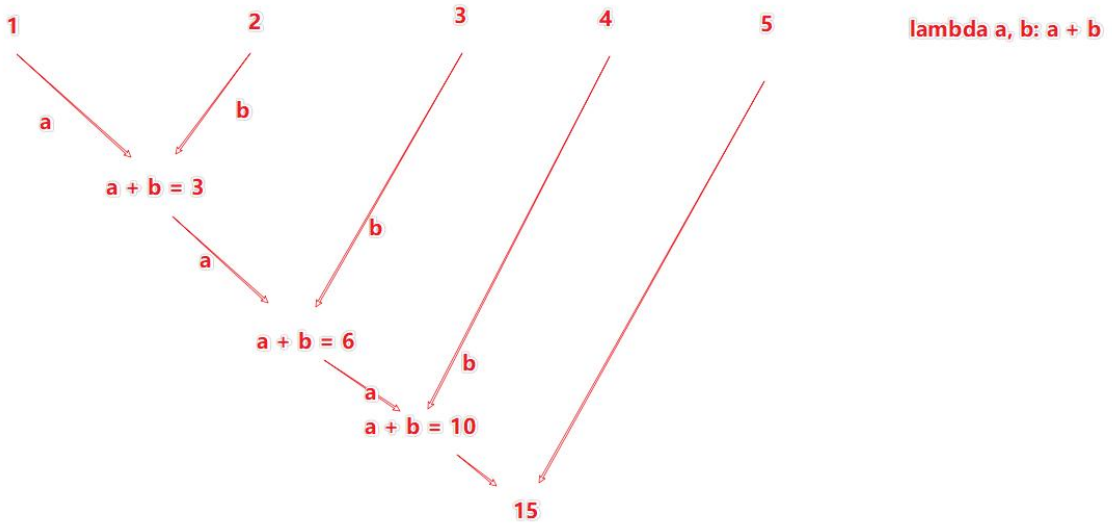
功能: 针对KV型 RDD, 自动按照key分组, 然后根据你提供的聚合逻辑, 完成 组内数据(value) 的聚合操作.

用法:

```
rdd.reduceByKey(func)
# func: (V, V) → V
# 接受2个传入参数(类型要一致), 返回一个返回值, 类型和传入要求一致.
```

reduceBeKey中的聚合逻辑是:

比如, 有[1, 2, 3, 4 ,5], 然后聚合函数是: `lambda a, b: a + b`



代码:

```
rdd = sc.parallelize([('a', 1), ('a', 1), ('b', 1), ('b', 1), ('b', 1)])
result = rdd.reduceByKey(lambda a, b: a + b)
print(result.collect())
# 结果:
[('b', 3), ('a', 2)]
```

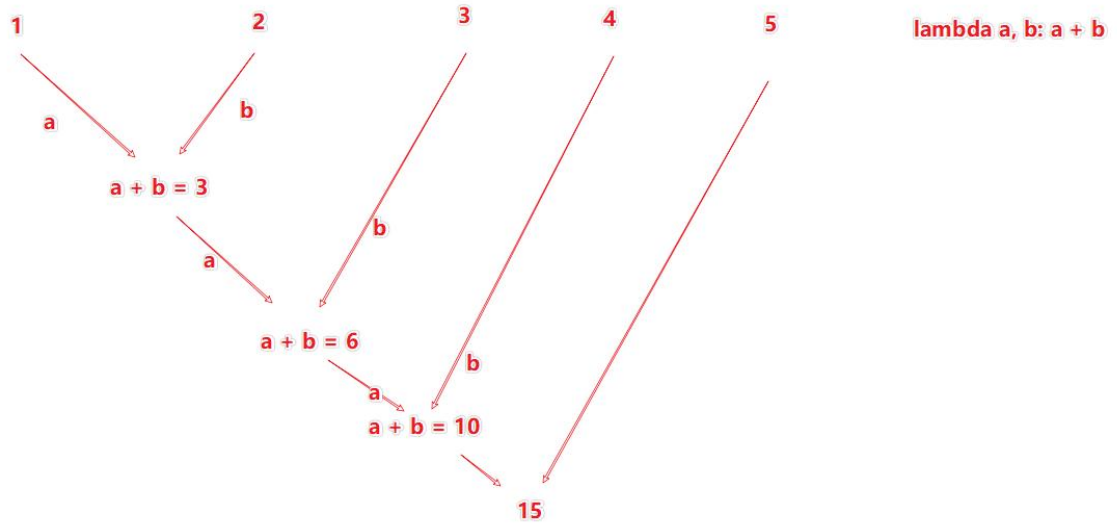
注意: reduceByKey中接收的函数, 只负责聚合, 不理会分组

分组是自动 `by key` 来分组的.

总结

1. reduceByKey算子

- 接受一个处理函数，对数据进行两两计算



目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 完成使用PySpark进行单词计数的案例

WordCount案例

使用学习到的内容，完成：

- 读取文件
- 统计文件内，单词的出现数量

```
# 导入pyspark的相关包，导入SparkConf，SparkContext
from pyspark import SparkConf, SparkContext


# 1. 构建Spark环境的执行入口对象 SparkContext对象
conf = SparkConf().setAppName("HelloSpark").\
    setMaster("local[*]")       # 设置Spark的Master
sc = SparkContext(conf = conf)  # sc 就是SparkContext对象，作为执行环境入口

# 2. 读取数据文件
file = sc.textFile("../data/words.txt", 3)

# 3. 将所有的单词都取出来
words = file.flatMap(lambda line: line.split(' '))

# 4. 将所有单词都加上1作为value
word_one = words.map(lambda x:(x, 1))

# 5. 分组并求和
result = word_one.reduceByKey(lambda a, b: a + b)

# 打印输出计算结果
print(result.collect())
```

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的filter方法

Filter

功能: 过滤想要的数据进行保留

语法:

```
rdd.filter(func)
# func: (T) → bool   传入1个参数进来随意类型，返回值必须是True or False
```

> 返回是True的数据被保留, False的数据被丢弃

示例代码:

```
# coding:utf8
# 演示rdd的filter算子

from pyspark import SparkConf, SparkContext

if __name__ == '__main__':
    # 0. 构建Spark执行环境
    conf = SparkConf().setAppName("create rdd"). \
        setMaster("local[*]")
    sc = SparkContext(conf=conf)

    rdd = sc.parallelize([1, 2, 3, 4, 5])

    # 保留奇数
    # rdd.filter(lambda x: True if (x % 2 == 1) else False)
    print(rdd.filter(lambda x: x % 2 == 1).collect())
```

总结

1. filter算子

- 接受一个处理函数，可用lambda快速编写
- 函数对RDD数据逐个处理，得到True的保留至返回值的RDD中

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的distinct方法

distinct算子

功能: 对RDD数据进行去重, 返回新RDD

语法:
rdd.distinct()　　无需传参

演示代码:

```
# coding:utf8
# 演示rdd的distinct算子

from pyspark import SparkConf, SparkContext

if __name__ == '__main__':
    # 0. 构建Spark执行环境
    conf = SparkConf().setAppName("create rdd"). \
        setMaster("local[*]")
    sc = SparkContext(conf=conf)

    rdd = sc.parallelize([1, 1, 3, 3, 5, 5, 6, 6, 9, 9])

    # 按照空格切分数据后，解除嵌套
    print(rdd.distinct().collect())
```

总结

1. distinct算子

- 完成对RDD内数据的去重操作

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握RDD的sortBy方法

sortBy算子

功能: 对RDD数据进行排序, 基于你指定的排序依据.

语法:

```
rdd.sortBy(func, ascending=False, numPartitions=1)
# func: (T) → U: 告知按照rdd中的哪个数据进行排序, 比如 lambda x: x[1] 表示按照rdd中的第二列元素进行排序
# ascending True升序 False 降序
# numPartitions: 用多少分区排序
```

总结

1. sortBy算子

- 接收一个处理函数，可用lambda快速编写
- 函数表示用来决定排序的依据
- 可以控制升序或降序
- 全局排序需要设置分区数为1

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
  - map方法
  - flatMap方法
  - reduceByKey方法
  - 练习案例1
  - filter方法
  - distinct方法
  - sortBy方法
  - 练习案例2
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 完成练习案例2的开发

案例

{"id":1,"timestamp":"2019-05-08T01:03.00Z","category":"平板电脑","areaName":"北京","money":"1450"}|{"id":2,"timestamp":"2019-05-08T01:01.00Z","category":"手机","areaName":"北京","money":"1450"}|{"id":3,"timestamp":"2019-05-08T01:03.00Z","category":"手机","areaName":"北京","money":"8412"}
{"id":4,"timestamp":"2019-05-08T05:01.00Z","category":"电脑","areaName":"上海","money":"1513"}|{"id":5,"timestamp":"2019-05-08T01:03.00Z","category":"家电","areaName":"北京","money":"1550"}|{"id":6,"timestamp":"2019-05-08T01:01.00Z","category":"电脑","areaName":"杭州","money":"1550"}
{"id":7,"timestamp":"2019-05-08T01:03.00Z","category":"电脑","areaName":"北京","money":"5611"}|{"id":8,"timestamp":"2019-05-08T03:01.00Z","category":"家电","areaName":"北京","money":"4410"}|{"id":9,"timestamp":"2019-05-08T01:03.00Z","category":"家具","areaName":"郑州","money":"1120"}
{"id":10,"timestamp":"2019-05-08T01:01.00Z","category":"家具","areaName":"北京","money":"6661"}|{"id":11,"timestamp":"2019-05-08T05:03.00Z","category":"家具","areaName":"杭州","money":"1230"}|{"id":12,"timestamp":"2019-05-08T01:01.00Z","category":"书籍","areaName":"北京","money":"5550"}
{"id":13,"timestamp":"2019-05-08T01:03.00Z","category":"书籍","areaName":"北京","money":"5550"}|{"id":14,"timestamp":"2019-05-08T01:01.00Z","category":"电脑","areaName":"北京","money":"1261"}|{"id":15,"timestamp":"2019-05-08T03:03.00Z","category":"电脑","areaName":"杭州","money":"6660"}
{"id":16,"timestamp":"2019-05-08T01:01.00Z","category":"电脑","areaName":"天津","money":"6660"}|{"id":17,"timestamp":"2019-05-08T01:03.00Z","category":"书籍","areaName":"北京","money":"9000"}|{"id":18,"timestamp":"2019-05-08T05:01.00Z","category":"书籍","areaName":"北京","money":"1230"}
{"id":19,"timestamp":"2019-05-08T01:03.00Z","category":"电脑","areaName":"杭州","money":"5551"}|{"id":20,"timestamp":"2019-05-08T01:01.00Z","category":"电脑","areaName":"北京","money":"2450"}
{"id":21,"timestamp":"2019-05-08T01:03.00Z","category":"食品","areaName":"北京","money":"5520"}|{"id":22,"timestamp":"2019-05-08T01:01.00Z","category":"食品","areaName":"北京","money":"6650"}
{"id":23,"timestamp":"2019-05-08T01:03.00Z","category":"服饰","areaName":"杭州","money":"1240"}|{"id":24,"timestamp":"2019-05-08T01:01.00Z","category":"食品","areaName":"天津","money":"5600"}
{"id":25,"timestamp":"2019-05-08T01:03.00Z","category":"食品","areaName":"北京","money":"7801"}|{"id":26,"timestamp":"2019-05-08T01:01.00Z","category":"服饰","areaName":"北京","money":"9000"}
{"id":27,"timestamp":"2019-05-08T01:03.00Z","category":"服饰","areaName":"杭州","money":"5600"}|{"id":28,"timestamp":"2019-05-08T01:01.00Z","category":"食品","areaName":"北京","money":"8000"}|{"id":29,"timestamp":"2019-05-08T02:03.00Z","category":"服饰","areaName":"杭州","money":"7000"}

需求，复制以上内容到文件中，使用Spark读取文件进行计算：

- 各个城市销售额排名，从大到小
- 全部城市，有哪些商品类别在售卖
- 北京市有哪些商品类别在售卖

案例

```python
# coding:utf8
# 演示rdd的算子案例1

from pyspark import SparkConf, SparkContext
import json

if __name__ == '__main__':
    # 0. 构建Spark执行环境
    conf = SparkConf().setAppName("create rdd"). \
        setMaster("local[*]")
    sc = SparkContext(conf=conf)

    # 需求: 读取data文件夹中的 order.text文件, 提取北京的数据, 组合北京和商品类别进行输出

    # 1. 读取文件
    rdd = sc.textFile("../data/order.text")

    # 2. 将每条json数据取出
    json_str_rdd = rdd.flatMap(lambda x: x.split("|"))

    # 3. 将每个json字符串, 变成字典对象
    json_dict_rdd = json_str_rdd.map(lambda x: json.loads(x))

    # 4. 过滤数据, 只保留北京
    beijing_rdd = json_dict_rdd.filter(lambda x: x['areaName'] == "北京")

    # 5. 组合北京 和 商品类型
    result_rdd = beijing_rdd.map(lambda x: x['areaName'] + "_" + x['category'])

    print(result_rdd.collect())
```

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
  - 输出为Python对象
  - 输出到文件中
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握将RDD的结果输出为Python对象的各类方法

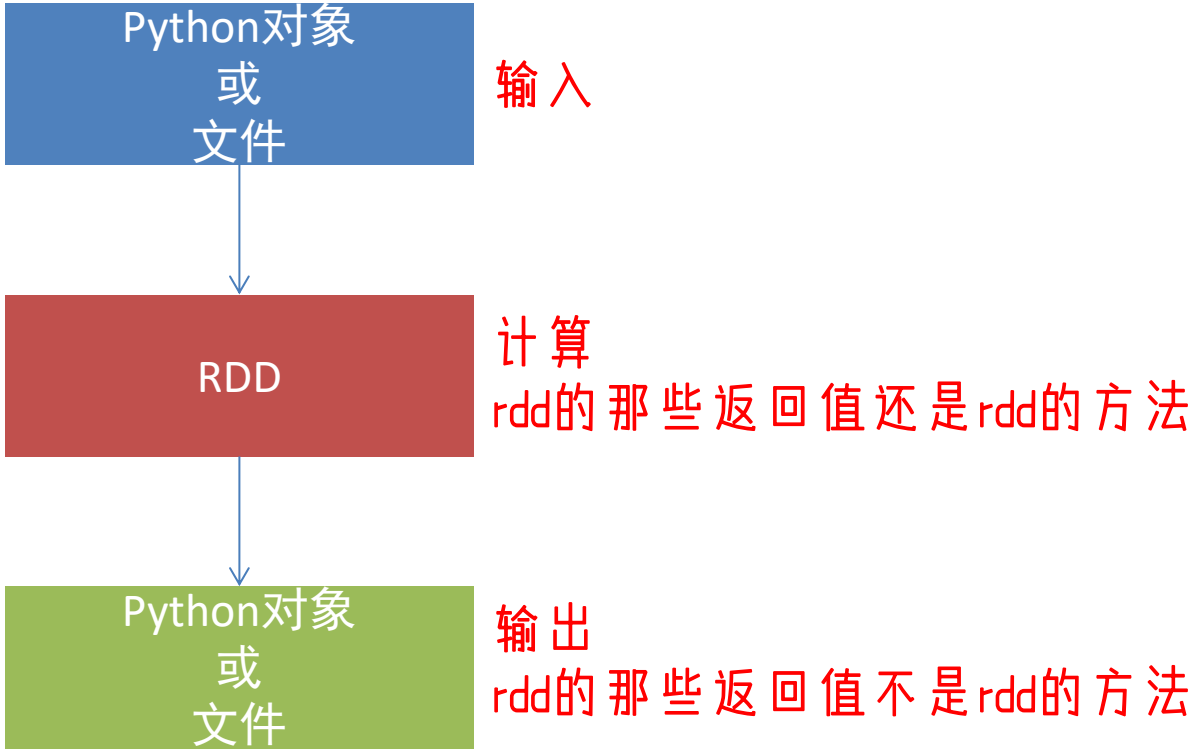
数据输出

数据输入：

- sc.parallelize
- sc.textFile

数据计算：

- rdd.map
- rdd.flatMap
- rdd.reduceByKey
- ...



collect算子

功能: 将RDD各个分区内的数据, 统一收集到Driver中, 形成一个List对象

用法:

```
rdd.collect()
```

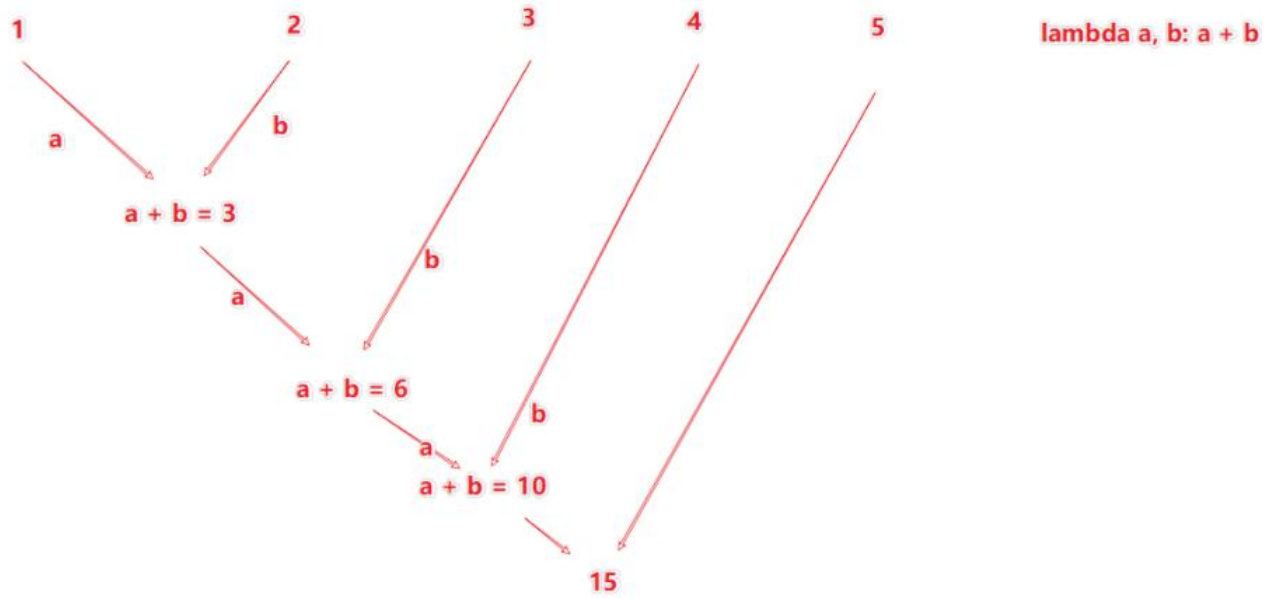
返回值是一个list

reduce算子

功能: 对RDD数据集按照你传入的逻辑进行聚合

语法:

```
rdd.reduce(func)
# func: (T, T) → T
# 2参数传入 1个返回值，返回值和参数要求类型一致.
执行流程如下图:
```



代码:

```
rdd = sc.parallelize(range(1, 10))
# 将rdd的数据进行累加求和
print(rdd.reduce(lambda a, b: a + b))
```

返回值等同于计算函数的返回值

take算子

功能: 取RDD的前N个元素, 组合成list返回给你

用法:

```
>>> sc.parallelize([3,2,1,4,5,6]).take(5)
[3, 2, 1, 4, 5]
```

count算子

功能: 计算RDD有多少条数据, 返回值是一个数字

用法:

```
>>> sc.parallelize([3,2,1,4,5,6]).count()
6
```

总结

1. Spark的编程流程就是：

- 将数据加载为RDD（数据输入）
- 对RDD进行计算（数据计算）
- 将RDD转换为Python对象（数据输出）

2. 数据输出的方法

- collect：将RDD内容转换为list
- reduce：对RDD内容进行自定义聚合
- take：取出RDD的前N个元素组成list
- count：统计RDD元素个数

数据输出可用的方法是很多的，本小节简单的介绍了4个。

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
  - 输出为Python对象
  - 输出到文件中
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 掌握将RDD的内容输出到文件中
2. 了解如何更改RDD的分区数为1

saveAsTextFile算子

功能: 将RDD的数据写入文本文件中

支持 本地写出, hdfs等文件系统.

代码:

```
rdd = sc.parallelize([1, 2, 3, 4, 5])
rdd.saveAsTextFile("../data/output/test.txt")
```

注意事项

调用保存文件的算子，需要配置Hadoop依赖

- 下载Hadoop安装包
  - http://archive.apache.org/dist/hadoop/common/hadoop-3.0.0/hadoop-3.0.0.tar.gz
- 解压到电脑任意位置
- 在Python代码中使用os模块配置：os.environ[‘HADOOP_HOME’] = ‘HADOOP解压文件夹路径’
- 下载winutils.exe，并放入Hadoop解压文件夹的bin目录内
  - https://raw.githubusercontent.com/steveloughran/winutils/master/hadoop-3.0.0/bin/winutils.exe
- 下载hadoop.dll，并放入:C:/Windows/System32 文件夹内
  - https://raw.githubusercontent.com/steveloughran/winutils/master/hadoop-3.0.0/bin/hadoop.dll

修改rdd分区为1个

方式1，SparkConf对象设置属性全局并行度为1：

```
conf = SparkConf().setMaster("local[*]").setAppName("test_spark")
conf.set("spark.default.parallelism", "1")
sc = SparkContext(conf=conf)
```

方式2，创建RDD的时候设置（parallelize方法传入numSlices参数为1）

```
rdd1 = sc.parallelize([1, 2, 3, 4, 5], numSlices=1)
```

```
rdd1 = sc.parallelize([1, 2, 3, 4, 5], 1)
```

总结

1. RDD输出到文件的方法

- rdd.saveAsTextFile(路径)
- 输出的结果是一个文件夹
- 有几个分区就输出多少个结果文件

2. 如何修改RDD分区

- SparkConf对象设置conf.set("spark.default.parallelism", "1")
- 创建RDD的时候，sc.parallelize方法传入numSlices参数为1

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

Learning Objectives

1. 完成综合案例的开发
2. 掌握\符号完成代码跨行编写

搜索引擎日志分析

00:00:00	2982199073774412	传智播客	8	3	http://www.itcast.cn
00:00:00	07594220010824798	黑马程序员	1	1	http://www.itcast.cn
00:00:00	5228056822071097	传智播客	14	5	http://www.itcast.cn
00:00:00	6140463203615646	博学谷	62	36	http://www.itcast.cn
00:00:00	8561366108033201	IDEA	3	2	http://www.itcast.cn
00:00:00	23908140386148713	传智专修学院	1	2	http://www.itcast.cn
23:00:00	1797943298449139	flume	8	5	http://www.itcast.cn
23:00:00	00717725924582846	itcast	1	2	http://www.itcast.cn
23:00:00	41416219018952116	bigdata	2	6	http://www.itcast.cn
23:00:00	9975666857142764	IDEA	2	2	http://www.itcast.cn
23:00:00	21603374619077448	酷丁鱼	1	6	http://www.itcast.cn
23:00:00	7423866288265172	bigdata	3	13	http://www.itcast.cn
23:00:00	0616877776407358	itcast	2	9	http://www.itcast.cn
23:00:00	3933365481995287	flume	6	3	http://www.itcast.cn
23:00:00	8242389147671512	数据仓库	2	3	http://www.itcast.cn
23:00:00	8248403977107859	传智汇	1	1	http://www.itcast.cn
23:00:00	6239728533154822	itheima	7	9	http://www.itcast.cn
23:00:00	6551182914925117	itcast	6	4	http://www.itcast.cn
23:00:00	2345161545038265	传智汇	2	1	http://www.itcast.cn
23:00:00	06478452674857782	hadoop	4	1	http://www.itcast.cn
23:00:00	23405445793591878	博学谷	4	5	http://www.itcast.cn
23:00:00	23457845712802688	hadoop	4	1	http://www.itcast.cn
23:00:00	4625224675315291	spark	2	6	http://www.itcast.cn
23:00:00	048731119953599966	itheima	10	44	http://www.itcast.cn
23:00:00	48530765688455246	hadoop	26	3	http://www.itcast.cn
23:00:00	5750662932822584	酷丁鱼	3	2	http://www.itcast.cn
23:00:00	11515839301781111	spark	4	3	http://www.itcast.cn
23:00:00	26706283607513126	java	5	6	http://www.itcast.cn
23:00:00	01532039495118448	hadoop	1	1	http://www.itcast.cn
23:00:01	0014362172758659586	hadoop	64	21	http://www.itcast.cn
23:00:01	8958844460084823	itcast	6	3	http://www.itcast.cn
23:00:01	14918066497166943	酷丁鱼	4	1	http://www.itcast.cn

读取文件转换成RDD，并完成：

- 打印输出：热门搜索时间段（小时精度）Top3
- 打印输出：热门搜索词Top3
- 打印输出：统计黑马程序员关键字在哪个时段被搜索最多
- 将数据转换为JSON格式，写出为文件

目录

Contents

- 前言介绍
- 基础准备
- 数据输入
- 数据计算
- 数据输出
- 综合案例
- 分布式集群运行

学习目标

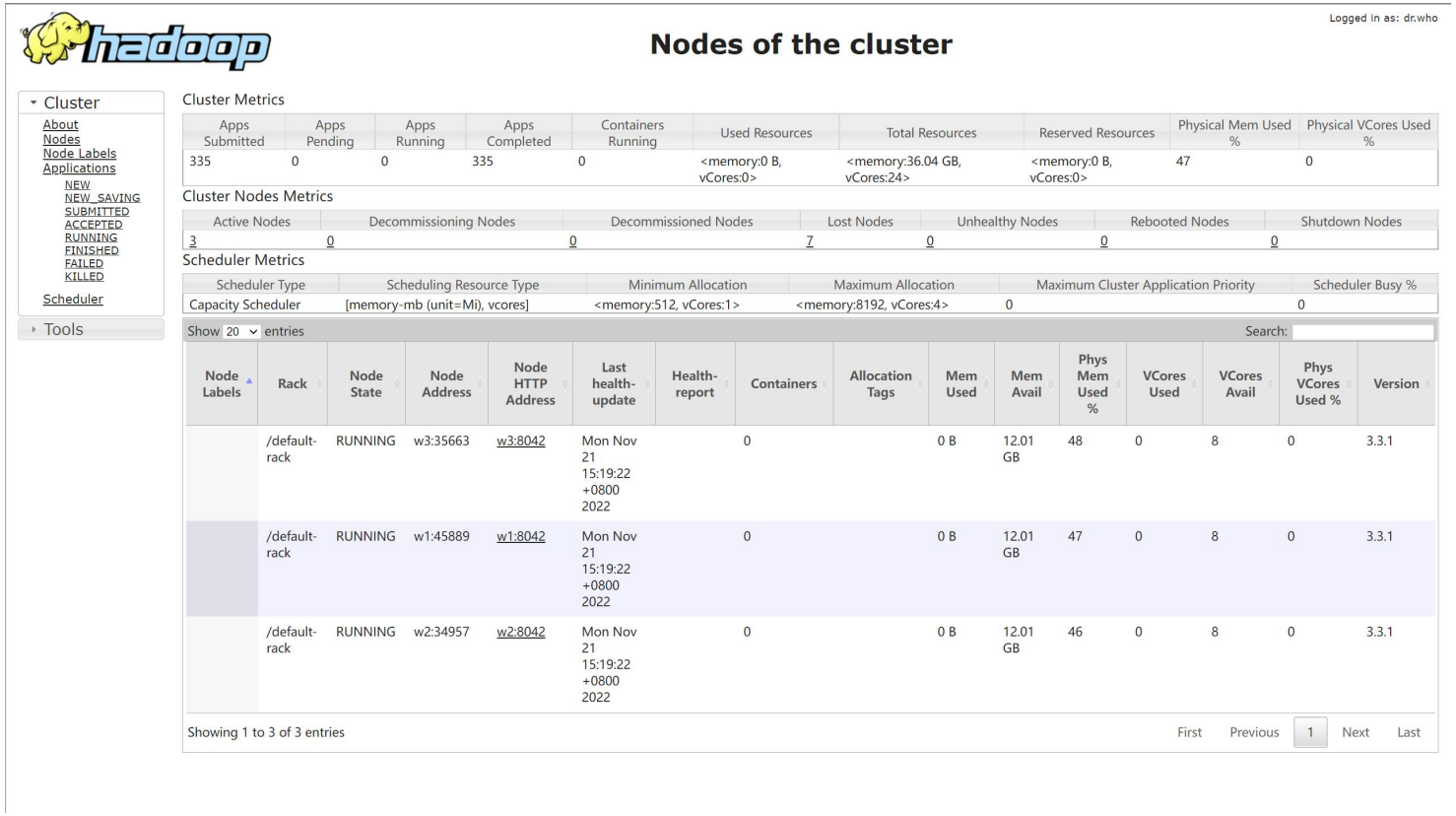
Learning Objectives

1. 了解PySpark代码在大数据集群上运行

将案例提交到YARN集群中运行

提交命令：

```
bin/spark-submit --master yarn --num-executors 3 --queue root.teach --executor-cores 4 --executor-memory 4g /home/hadoop/demo.py
```



黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌